

CLI y Bash a través de Git

Bash



Objetivos de Aprendizaje

- Comprender qué es la línea de comandos (CLI) y su importancia
- Familiarizarse con Git Bash como terminal en Windows
- Aprender comandos básicos de navegación y manipulación de archivos
- Aplicar comandos Bash en el contexto del desarrollo web



¿Qué es la Línea de Comandos?

- **CLI** = Command Line Interface (Interfaz de Línea de Comandos)
- Es una forma de interactuar con el ordenador usando solo texto
- Alternativa a la interfaz gráfica (GUI) que usamos habitualmente
- **Ventajas:**
 - Más rápida para ciertas tareas
 - Permite automatización
 - Fundamental para el desarrollo web
 - Acceso a herramientas avanzadas



Git Bash: bash en Windows

- Windows tradicionalmente usa **Command Prompt** (cmd) y **PowerShell**
- Git Bash proporciona un entorno **Unix-like** en Windows
- Ventajas de Git Bash:
 - Comandos similares a Linux/Mac (estándar en desarrollo web)
 - Incluye Git integrado
 - Sintaxis Bash (más amigable que cmd)
 - Colores y autocompletado

Comparativa	
Windows CMD:	'dir'
Git Bash:	'ls'



Cómo Acceder a Git Bash

Método 1: Desde el menú de inicio

1. Presiona tecla Windows
2. Escribe "Git Bash"
3. Enter

Método 2: Desde cualquier carpeta

1. Navega a la carpeta en el explorador
2. Click derecho en espacio vacío
3. Selecciona "Git Bash Here"

Método 3: Desde VS Code

1. Abre VS Code
2. Terminal -> New Terminal
3. Selecciona "Git Bash" en el desplegable



Anatomía del Terminal

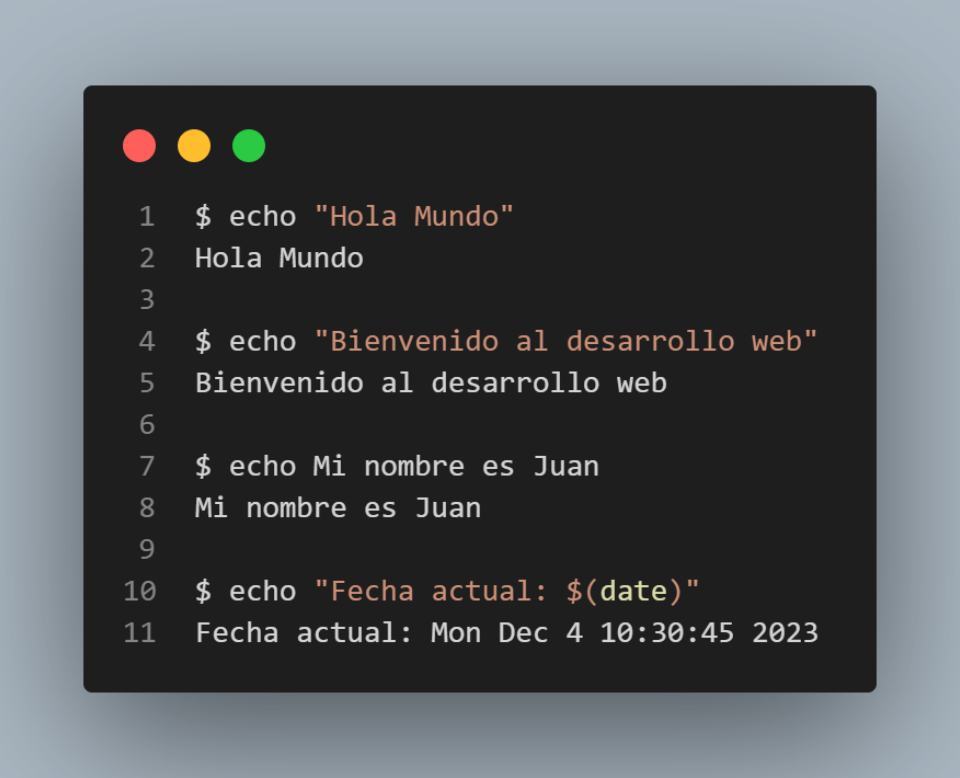
- Desglose
 - **MINGW64**: Entorno de emulación Unix
 - **/c/Users/TuNombre**: Directorio actual
 - **(master)**: Rama de Git actual (si estás en un repositorio)
 - **\$**: Prompt, indica que está listo para recibir comandos
- Elementos importantes
 - **Cursor**: donde aparecerá el texto que escribas
 - **Colores**: Ayudan a distinguir tipos de archivos y directorios



Tu Primer Comando: echo

- ¿Qué hace echo?
 - Muestra texto en pantalla
 - Útil para mostrar mensajes o contenido de variables

Nota: Las comillas son opcionales para texto simple, pero recomendadas para frases



```
1 $ echo "Hola Mundo"
2 Hola Mundo
3
4 $ echo "Bienvenido al desarrollo web"
5 Bienvenido al desarrollo web
6
7 $ echo Mi nombre es Juan
8 Mi nombre es Juan
9
10 $ echo "Fecha actual: $(date)"
11 Fecha actual: Mon Dec 4 10:30:45 2023
```

Navegación y Estructura de Directorios



Entendiendo la Estructura de Directorios

En Git Bash las rutas se escriben estilo Unix:

Windows: C:\Users\TuNombre\Documents

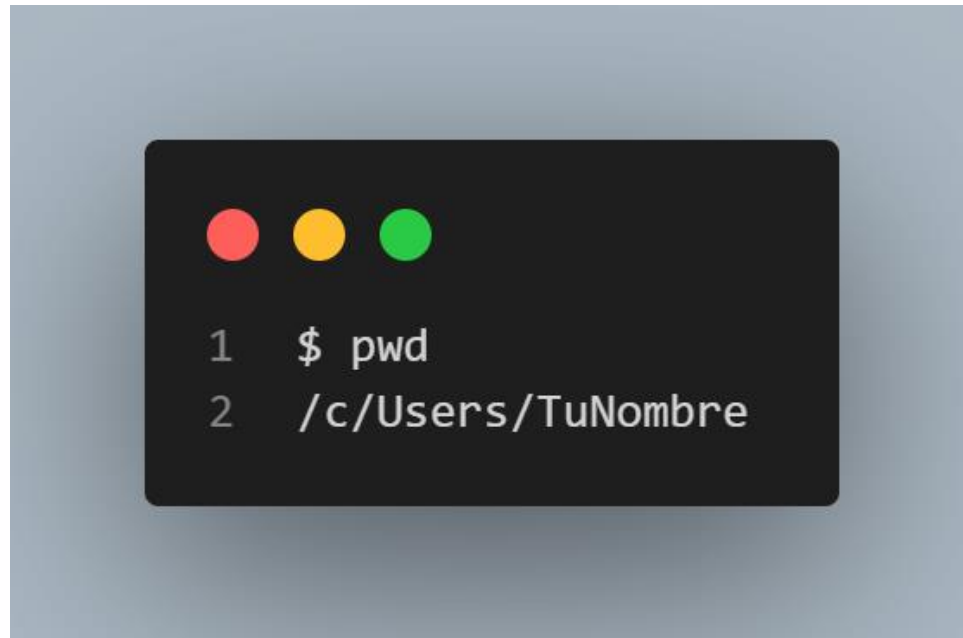
Git Bash: /c/Users/TuNombre/Documents

Directorios especiales:

- '/' = Raíz del sistema
- '~' = Tu directorio personal (/c/Users/TuNombre)
- '.' = Directorio actual
- '..' = Directorio padre (uno arriba)



'pwd' - Print Working Directory

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It shows a command prompt with two lines of input and output.

```
1 $ pwd
2 /c/Users/TuNombre
```

¿Qué hace pwd?

- **P**rint **W**orking **D**irectory
- Muestra la ruta completa del directorio actual
- Es como preguntar "¿dónde estoy ahora?"

Cuándo usarlo:

- Cuando te sientes perdido en el terminal
- Para confirmar tu ubicación antes de ejecutar comandos
- Al empezar a trabajar en el terminal



'ls' - Listar Contenido



```
1 $ ls
2 Desktop Documents Downloads Pictures
3
4 . # Variaciones útiles:
5 $ ls -l      # Lista detallada con permisos, tamaños, fechas
6 $ ls -a      # Muestra archivos ocultos (empiezan con .)
7 $ ls -la     # Combina ambas opciones
8 $ ls *.html  # Solo archivos HTML
9
10 . # Resultado de ls -la:
11 drwxr-xr-x 5 user group 160 Dec  4 10:30 .
12 drwxr-xr-x 3 user group  96 Dec  3 15:20 ..
13 -rw-r--r-- 1 user group 1024 Dec  4 09:15 index.html
```

'cd' - Change Directory



```
1 $ cd Documents      # Ir a la carpeta Documents
2 $ cd ..             # Subir un nivel
3 $ cd ~              # Ir al directorio personal
4 $ cd /              # Ir a la raíz del sistema
5
6 . # Navegación con rutas:
7 $ cd Documents/proyectos/mi-web    # Ruta relativa
8 $ cd /c/Users/TuNombre/Desktop    # Ruta absoluta
9
10 . # Truco útil:
11 $ cd -                          # Volver al directorio anterior
12
13 . # Autocompletado: Escribe `cd Doc` y presiona Tab para autocompletar
```

Manipulación de Archivos y Carpetas



'mkdir' - Make Directory

- La opción -p:
 - Crea directorios padre si no existen
 - Sin -p: error si el directorio padre no existe
 - Con -p: crea toda la estructura necesaria

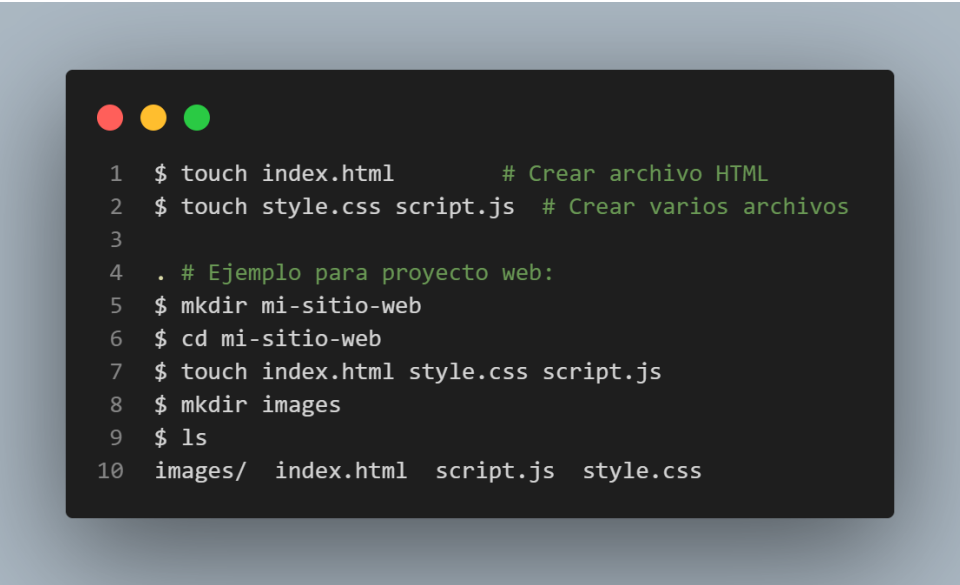
```
1 $ mkdir mi-proyecto          # Crear una carpeta
2 $ mkdir proyecto1 proyecto2 proyecto3 # Crear varias
3 $ mkdir -p proyectos/web/html # Crear estructura completa
4
5 . # Ejemplo práctico para desarrollo web:
6 $ mkdir -p mi-web/{css,js,images,html}
7 . # Crea: mi-web/css, mi-web/js, mi-web/images, mi-web/htm
```



'touch' - Crear Archivos Vacíos

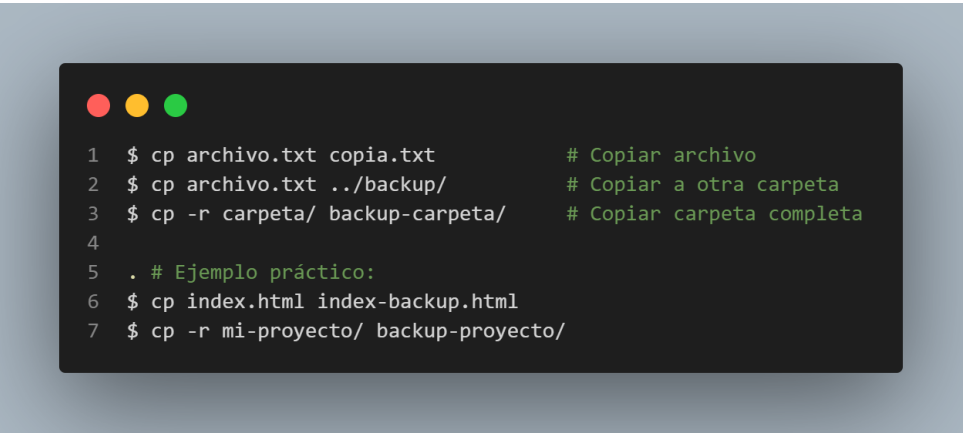
¿Qué hace touch?

- Crea un archivo vacío si no existe
- Si el archivo existe, actualiza su fecha de modificación
- Es la forma más rápida de crear archivos en terminal

A terminal window with a dark background and light green text. It shows a series of commands and their outputs. The commands are: 1. \$ touch index.html (comment: # Crear archivo HTML), 2. \$ touch style.css script.js (comment: # Crear varios archivos), 3. (blank line), 4. . # Ejemplo para proyecto web: (comment), 5. \$ mkdir mi-sitio-web, 6. \$ cd mi-sitio-web, 7. \$ touch index.html style.css script.js, 8. \$ mkdir images, 9. \$ ls, 10. images/ index.html script.js style.css (output).

```
1 $ touch index.html          # Crear archivo HTML
2 $ touch style.css script.js  # Crear varios archivos
3
4 . # Ejemplo para proyecto web:
5 $ mkdir mi-sitio-web
6 $ cd mi-sitio-web
7 $ touch index.html style.css script.js
8 $ mkdir images
9 $ ls
10 images/ index.html script.js style.css
```

'cp' - Copy

A terminal window with a dark background and light green text. It shows several examples of the 'cp' command. The first three lines show basic usage: copying a file, copying to a directory, and recursive copying. The fourth line is empty. The fifth line starts with a comment. The next two lines show recursive copying of a file and a directory. The terminal window has three colored dots (red, yellow, green) in the top left corner.

```
1 $ cp archivo.txt copia.txt           # Copiar archivo
2 $ cp archivo.txt ../backup/          # Copiar a otra carpeta
3 $ cp -r carpeta/ backup-carpeta/     # Copiar carpeta completa
4
5 . # Ejemplo práctico:
6 $ cp index.html index-backup.html
7 $ cp -r mi-proyecto/ backup-proyecto/
```

Opciones importantes:

- -r o -R: Recursivo, para copiar carpetas
- -i: Interactivo, pregunta antes de sobrescribir
- -v: Verbose, muestra qué se está copiando

'mv' - Mover y Renombrar

Diferencias con cp:

- 'cp': hace una copia, el original queda
- 'mv': mueve o renombra, el original desaparece

```
1 $ mv archivo-viejo.txt archivo-nuevo.txt # Renombrar
2 $ mv archivo.txt documentos/             # Mover a carpeta
3 $ mv *.html html/                        # Mover todos los HTML
4
5 . # Ejemplos comunes en desarrollo web:
6 $ mv index.htm index.html                # Corregir extensión
7 $ mv css/*.css styles/                   # Reorganizar archivos CSS
8 $ mv temp-project/ my-website/           # Renombrar proyecto
```




- 'rm' elimina permanentemente (no va a la papelera)
- 'rm -fr' es muy peligroso (fuerza eliminación)
- Siempre usa '-i' para confirmación cuando aprendas
- Haz backup antes de eliminar cosas importantes

```
1 $ rm archivo.txt # Eliminar archivo
2 $ rm -i archivo.txt # Preguntar antes de eliminar
3 $ rm -r carpeta/ # Eliminar carpeta y contenido
4 $ rm *.tmp # Eliminar archivos temporales
5
6 . # Buenas prácticas:
7 $ rm -i *.log # Siempre confirmar
8 $ ls # Verificar qué hay antes de eliminar
```



'cat' - Concatenate/Mostrar Archivos



```
1 $ cat index.html          # Mostrar contenido completo
2 $ cat style.css script.js # Mostrar varios archivos
3
4 . # Comandos relacionados útiles:
5 $ head index.html         # Primeras 10 líneas
6 $ tail script.js          # Últimas 10 líneas
7 $ less style.css          # Ver archivo página por página
8
9 . # Uso en desarrollo web:
10 $ cat package.json        # Ver configuración de proyecto
11 $ head -20 index.html     # Ver inicio de archivo HTML
```



Redirección: Guardar Salidas

Símbolos de redirección:

- '>': Redirige y sobrescribe
- '>>': Redirige y añade al final
- '|': Pipe, pasa salida al siguiente comando

```
1 $ ls > lista-archivos.txt      # Guardar lista en archivo
2 $ echo "Hola" >> saludo.txt    # Añadir al final del archivo
3 $ cat *.html > todo.html      # Combinar archivos HTML
4 ```
5
6 **Símbolos de redirección:**
7 - `>`: Redirige y sobrescribe
8 - `>>`: Redirige y añade al final
9 - `|`: Pipe, pasa salida al siguiente comando
10
11 **Ejemplos útiles:**
12 ```bash
13 $ ls > lista-archivos.txt      # Guardar lista en archivo
14 $ echo "Hola" >> saludo.txt    # Añadir al final del archivo
15 $ cat *.html > todo.html      # Combinar archivos HTML
16
17 . # Ejemplos útiles:
18 $ ls -la > inventario.txt      # Documentar contenido
19 $ find . -name "*.html" > html-files.txt # Lista de HTML
```

Práctica: Creando Estructura de Proyecto



```
1  . # 1. Crear estructura de directorios
2  $ mkdir mi-sitio-web
3  $ cd mi-sitio-web
4  $ mkdir css js images docs
5
6  . # 2. Crear archivos principales
7  $ touch index.html about.html contact.html
8  $ touch css/style.css css/responsive.css
9  $ touch js/main.js js/utils.js
10
11 . # 3. Verificar estructura
12 $ ls -la
13 $ tree    # Si tienes tree instalado
14
15 . # 4. Crear archivo de documentación
16 $ echo "# Mi Sitio Web" > README.md
17 $ echo "Proyecto creado el $(date)" >> README.md
```



kingscorner